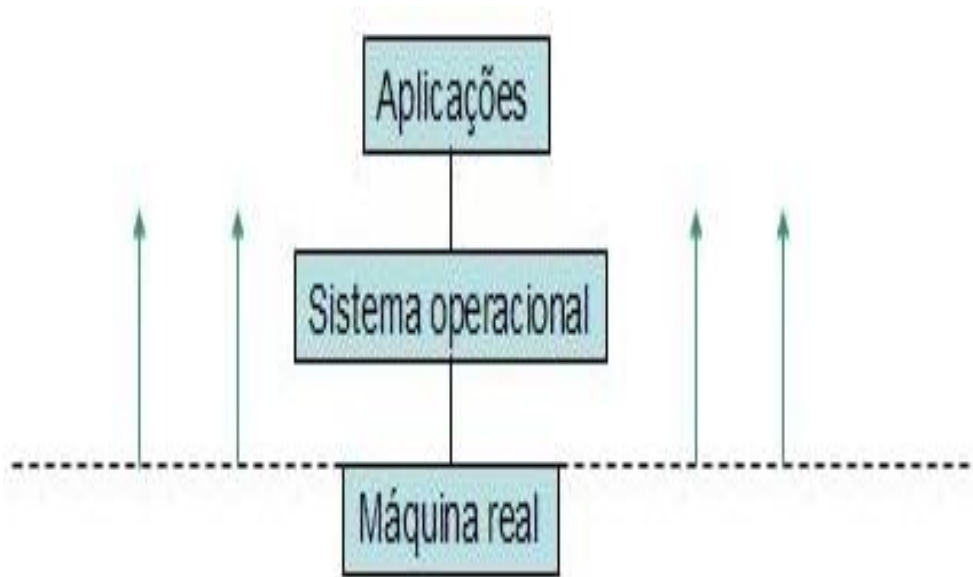


Programação Orientada a Objetos em Java

Paulo Henrique

Máquina Virtual

- Implementação através de software de uma máquina capaz de executar programas.
- Máquinas virtuais simulam a execução de sistemas operacionais.

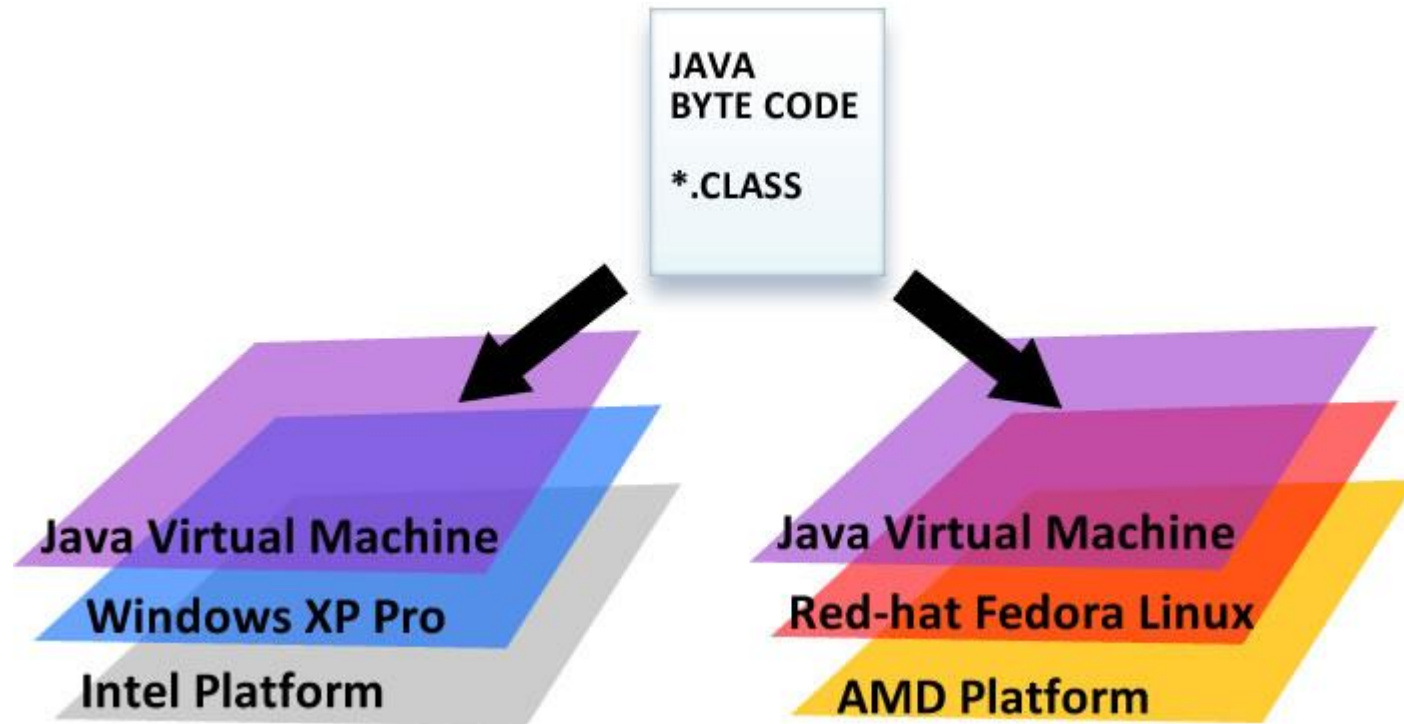


Máquina Virtual Java (JVM)

- Um programa para executar programas em Java.
- Programas em Java contém instruções em *bytecodes*.

Bytecode – Linguagem intermediária **compilada** por Java, podendo ser gerada por outras linguagens.

JVM



Processo de Execução

Fase 1: Edita

Editor



O programa é criado em um editor e armazenado no disco em um arquivo, com a extensão .java.

Fase 2: Compila

Compilador



O compilador cria bytecodes e os armazena no disco em um arquivo com a extensão .class.

Fase 3: Carrega

Class Loader



Memória Principal

O carregador de classe lê arquivos .class que contém bytecodes a partir de um disco e coloca esses bytecodes na memória.

Fase 4: Verifica

Verificador de Bytecode

Memória Principal

O verificador de bytecodes confirma que todos os bytecodes são válidos e não violam restrições de segurança do Java.

Fase 5: Executa

JVM

Memória Principal

Para executar o programa, o JVM lê os bytecodes e os traduz para uma linguagem que o computador possa entender. Assim que o programa é executado, é possível armazenar os valores dos dados em uma memória principal.

Estrutura de um programa Java

- Um programa é dividido em partes chamadas **classes**.
- As classes também são divididas em partes, chamadas **métodos. (Funções/Proc. em C++)**.
- Tais métodos são blocos de códigos que podem ou não retornar dados no fim de sua execução.

Observação

- O programador pode criar cada classe que precisar para fazer programas em Java.
- Java possui coleções de classes armazenadas em Bibliotecas, conhecidas como **APIs**.

Primeiro programa em Java

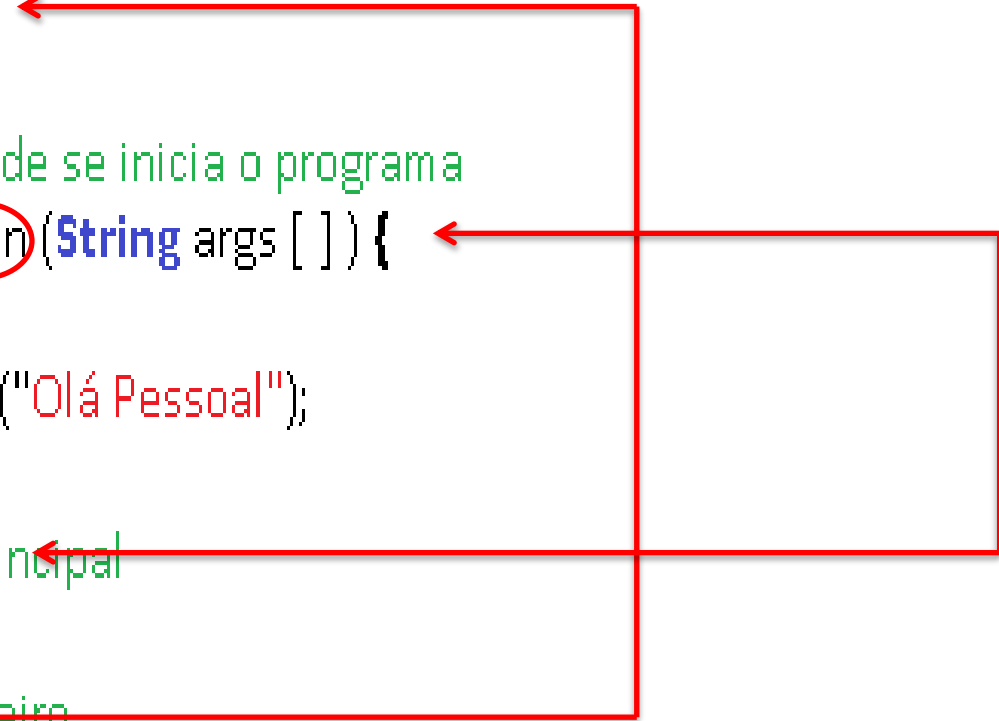
```
1. //Programa para imprimir texto
2.
3. public class Primeiro {
4.
5. //método principal por onde se inicia o programa
6.     public static void main (String args [ ] ) {
7.
8.         System.out.println ("Olá Pessoal");
9.
10.    } //Fim do método principal
11.
12 } //Fim da Classe Primeiro
13.
```

Primeiro programa em Java

```
1. //Programa para imprimir texto    Comentários em Java - // ou /**/  
2.   
3. public class Primeiro {  
4.  
5. //método principal por onde se inicia o programa  
6.     public static void main (String args [ ] ) {  
7.  
8.         System.out.println ("Olá Pessoal");  
9.  
10.    } //Fim do método principal  
11.  
12 } //Fim da Classe Primeiro  
13.
```

Primeiro programa em Java

```
1. //Programa para imprimir texto
2.
3. public class Primeiro {
4.
5. //método principal por onde se inicia o programa
6.     public static void main(String args [ ] ) {
7.
8.         System.out.println ("Olá Pessoal");
9.
10.    } //Fim do método principal
11.
12. } //Fim da Classe Primeiro
13.
```



The diagram consists of red lines and arrows illustrating the execution flow of the Java program. A red line starts from the right side of line 6, goes right, then down, then left, ending with an arrow pointing to line 3. Another red line starts from the right side of line 10, goes right, then down, then left, ending with an arrow pointing to line 12. A third red line starts from the right side of line 12, goes right, then down, then left, ending with an arrow pointing to line 3. These lines represent the sequence of execution: starting at the main method (line 6), jumping to the class definition (line 3), executing the class body (lines 8-9), returning to the main method (line 10), and finally exiting the class (line 12).

Primeiro programa em Java

```
1. //Programa para imprimir texto
2.
3. public class Primeiro {
4.
5. //método principal por onde se inicia o programa
6.     public static void main (String args [ ] ) {
7.
8.         System.out.println("Olá Pessoal");
9.
10.    } //Fim do método principal
11.
12 } //Fim da Classe Primeiro
13.
```

Modificando o 1º programa...

- `System.out.print` x `System.out.println`
- Caracteres de escape...

Escape sequence	Description
<code>\n</code>	Newline. Position the screen cursor to the beginning of the next line.
<code>\t</code>	Horizontal tab. Move the screen cursor to the next tab stop.
<code>\r</code>	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
<code>\a</code>	Alert. Sound the system bell.
<code>\\</code>	Backslash. Used to print a backslash character.
<code>\'</code>	Single quote. Use to print a single quote character.
<code>\"</code>	Double quote. Used to print a double quote character.

Inserindo dados

```
3 import java.util.Scanner; // programa utiliza a classe Scanner
4
5 public class Addition
6 {
7     // método principal inicia a execução do aplicativo Java
8     public static void main( String args[] )
9     {
10         // cria Scanner para obter entrada a partir da janela de comando
11         Scanner input = new Scanner( System.in );
12
13         int number1; // primeiro número a somar
14         int number2; // segundo número a somar
15         int sum; // soma de number1 e number2
16
17         System.out.print( "Enter first integer: " ); // prompt
18         number1 = input.nextInt(); // lê o primeiro número fornecido pelo usuário
19
20         System.out.print( "Enter second integer: " ); // prompt
21         number2 = input.nextInt(); // lê o segundo número fornecido pelo usuário
22
23         sum = number1 + number2; // soma os números
24
25         System.out.printf( "Sum is %d\n", sum ); // exibe a soma
26
27     } // fim do método principal
28
29 } // fim da classe Addition
```